



poll()



Polling without non-blocking sockets still blocks.

`poll()` is a system call used to wait for events on multiple file descriptors, allowing an application to monitor multiple input/output sources efficiently.

```
#include <poll.h>

struct pollfd {
    int    fd;        // File descriptor to monitor
    short  events;     // Events to watch for (input)
    short  revents;    // Events that occurred (output)
};

int poll(struct pollfd *fds, nfds_t nfds, int timeout);
```

- `fds`: Array of `pollfd` structures
- `nfds`: Number of items in array
- `timeout`: Milliseconds to wait (-1 = forever, 0 = non-blocking)

Returns the number of ready FDs, 0 if timeout, -1 on error.

Event flags:

- POLLIN: Data available to read
- POLLOUT: Ready to write
- POLLERR: Error occurred
- POLLHUP: Hang up (client disconnected)
- POLLNVAL: Invalid FD

<https://www.man7.org/linux/man-pages/man2/poll.2.html>

Warnings

Use a single poll() for all I/O:

```
// BAD - separate poll for CGI
poll(clientFds); // Main poll
poll(cgiPipeFds); // Another poll - FORBIDDEN

// GOOD - everything in one poll
std::vector<struct pollfd> allFds;
allFds.push_back(listenPfd); // Listening sockets
allFds.push_back(clientPfd); // Client sockets
allFds.push_back(cgiReadPfd); // CGI pipe (read end)
allFds.push_back(cgiWritePfd); // CGI pipe (write end)

poll(&allFds[0], allFds.size(), -1);
```

Never read/write without poll() readiness:

```
// BAD - reading without checking
recv(clientFd, buffer, size, 0); // May block!

// GOOD - only read when poll() says POLLIN
if (pfd.revents & POLLIN) {
    recv(pfd.fd, buffer, size, 0);
}
```

Monitor both reading and writing simultaneously:

```
// A client might need both
struct pollfd pfd;
pfd.fd = clientFd;
pfd.events = POLLIN | POLLOUT; // Watch for both
```

Not needed for regular files:

```
// OK to read/write files without poll()
int fileFd = open("index.html", O_RDONLY);
read(fileFd, buffer, 1024); // Allowed - regular file, not socket
```